Flasher une cible par LoRa : mode opératoire

PDF français · English PDF

English

Guide de la branche expérimentale `experiment/lora-ota`, vérifié contre `tools/lora_ota/send_integrated.py`. Les exemples concernent les TTGO LoRa32 V2.1 / SX1278 du banc. Ils ne constituent pas une procédure validée pour tous les matériels du flasher public.

La commande de transfert complet ci-dessous envoie l'image, la vérifie, l'active, puis confirme son démarrage automatiquement. Elle ne s'arrête pas pour demander une validation à l'opérateur entre ces étapes.

1. Ce qui doit être prêt

```
PC ── USB ── pont radio ── LoRa ── cible
                          ou
PC ── USB ── pont radio ── LoRa ── relais ── LoRa ── cible
```

Équipement	Rôle et prérequis
Pont USB	Firmware `bridge_fast`, protocole série du contrôleur à 921600 bauds. Il ne fournit pas le serveur TNC2, le Wi-Fi ni le traitement des commandes APRS d'un iGate normal.
Cible	Firmware compatible `ENABLE_RF_OTA`, deux partitions application et un bootloader compatibles avec le retour arrière, clé REMOTE_AUTH et indicatif du contrôleur provisionnés.
Relais éventuel	Firmware compatible `ENABLE_RF_ROUTE`, rôle relais et route provisionnés. La cible doit aussi être configurée pour ce relais.
Image à envoyer	Application `firmware.bin` compatible avec la carte, assez petite pour sa partition inactive. Le nouveau firmware doit conserver le service OTA radio et la confirmation différée.

Le transfert radio ne remplace ni le bootloader ni la table des partitions. La première installation compatible nécessite un accès préalable à la carte. Pendant le transfert, le programme n'ouvre que l'USB du pont : il n'utilise ni l'USB ni le Wi-

Fi de la cible.

Les champs binaires source, cible et relais sont limités à **huit caractères**. `--target` désigne le **Callsign-SSID configuré**, par exemple `F4MLV-15`, même si la carte se présente en APRS sous `GENATX`.

Pour le relayage, deux clés distinctes sont nécessaires : la clé de la cible pour l'authentification de bout en bout, et une clé de transport commune au contrôleur, au relais et à la cible. Toutes deux sont des clés de 32 octets représentées en base64url, soit 43 caractères sans remplissage.

2. Préparer le poste et les fichiers

Travailler dans un checkout durable déjà placé sur `experiment/lora-ota`. Vérifier son état avant de changer de branche ou de remplacer des fichiers. Les commandes de ce guide supposent Bash ou Zsh, PlatformIO, Python 3 et PySerial disponibles.

```
git status --short --branch
git log -3 --oneline
python3 -c 'import serial; print(serial.__version__)'
python3 tools/lora_ota/send_integrated.py --help
```

Définir les chemins et identités. Les ports ci-dessous sont à remplacer. Réutiliser les clés et le compteur existants si la chaîne est déjà préparée.

```
export OTA_BRIDGE='/dev/serial/by-id/PORT_DU_PONT'
export OTA_SOURCE='F4MLV-2'
export OTA_TARGET='F4MLV-15'
export OTA_VIA='F4MLV-10'
export OTA_PRIVATE='docs/local/ota'
export OTA_KEY="$OTA_PRIVATE/target.key"
export OTA_ROUTE_KEY="$OTA_PRIVATE/route.key"
export OTA_COUNTER="$OTA_PRIVATE/F4MLV-2--F4MLV-15.counter"
export OTA_PROFILES="$OTA_PRIVATE/profiles.json"
export OTA_IMAGE='.pio/build/ttgo-lora32-v21-ota-b/firmware.bin'
export OTA_RUN="logs/radio-ota-$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -p "$OTA_PRIVATE" "$OTA_RUN"
chmod 700 "$OTA_PRIVATE" "$OTA_RUN"
```

Les fichiers privés et les journaux ne doivent pas être publiés. Le compteur reste le même d'un transfert à l'autre pour cette paire contrôleur/cible ; le journal, lui, doit avoir un nouveau chemin à chaque lancement, car le client ouvre son fichier de log en écriture.

Clés : seulement lors du premier provisionnement

Si les clés sont déjà installées, **sauter ce bloc** et utiliser leurs fichiers existants. Il refuse de remplacer un fichier présent et n'affiche aucune clé.

```
python3 - <<'PY'
import os, secrets, base64
from pathlib import Path
for name in ('OTA_KEY', 'OTA_ROUTE_KEY'):
    path = Path(os.environ[name])
    key = base64.urlsafe_b64encode(secrets.token_bytes(32)).rstrip(b'=')
    fd = os.open(path, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
    with os.fdopen(fd, 'wb') as out:
        out.write(key)
PY
```

Profils radio

Copier le fichier d'exemple seulement s'il n'existe pas encore :

```
test -e "$OTA_PROFILES" || cp tools/lora_ota/radio-profiles.example.json "$OTA_PROFILES"
python3 -m json.tool "$OTA_PROFILES"
```

`normal` doit correspondre aux réglages **déjà enregistrés sur la cible et le relais**. L'exemple du dépôt utilise 434,100 MHz/BW125/SF12/CR4/5/2 dBm, qui est le profil du banc, pas le 433,775 MHz du réseau habituel. `transfer` contient le profil temporaire : l'exemple utilise 434,550 MHz/BW250/SF8/CR4/6/3 dBm.

Fréquence et bande passante sont en Hz, SF est entier, `coding_rate` est le dénominateur de 4/CR, et `power_dbm` la puissance demandée. Le client transmet ce profil par RF et règle le pont. Aucun `OTA=FAST` manuel n'est nécessaire avant le lancement.

3. Première installation, uniquement si nécessaire

Sauter cette section lorsque les cartes portent déjà le firmware du banc. Ne pas les reflasher entre les passages d'une même campagne.

Compiler les variantes nécessaires :

```
pio run -e ttgo-lora32-v21-ota-a -e ttgo-lora32-v21-ota-b -e ttgo-lora32-v21-ota-relay
pio run -d tools/lora_ota -e bridge_fast
```

Pour une TTGO à flash de 4 Mio, sauvegarder le contenu complet avant une installation USB, avec le port réel de la carte concernée :

```
export OTA_TARGET_USB='/dev/serial/by-id/PORT_USB_DE_LA_CIBLE'
esptool.py --chip esp32 --port "$OTA_TARGET_USB" --baud 921600 read_flash 0 0x400000
        "$OTA_RUN/target-before.bin"

sha256sum "$OTA_RUN/target-before.bin" > "$OTA_RUN/target-before.sha256"
```

Faire également les sauvegardes des configurations et des autres cartes qui seront remplacées par un firmware de banc. Ne pas supposer que la même taille de flash ou les mêmes offsets conviennent à un autre matériel.

Installation du pont et de la cible initiale :

```
pio run -d tools/lora_ota -e bridge_fast -t upload --upload-port "$OTA_BRIDGE"

pio run -e ttgo-lora32-v21-ota-a -t upload --upload-port "$OTA_TARGET_USB"
```

L'installation USB écrit aussi les artefacts de démarrage/partitions du variant. Vérifier leur compatibilité avant de remplacer une disposition existante. Les essais ont utilisé deux slots de 1 966 080 octets.

Provisionner la clé cible et le contrôleur dans l'interface de gestion du firmware. Les variantes de diagnostic offrent aussi ces commandes série, à 115200 bauds, avec les contenus réels des fichiers de clés :

```
python3 -m serial.tools.miniterm "$OTA_TARGET_USB" 115200 --raw
```

Quitter le moniteur avec `Ctrl+]` avant de lancer le transfert.

```
OTAPROVISION F4MLV-2 <clé cible base64url>
```

Réponse attendue : `OTAPROVISIONED`. Le choix du contrôleur est enregistré avec cette commande. Ne pas changer de clé sur une cible déjà préparée.

Pour exiger une cible sans infrastructure IP, la commande de diagnostic `OTAOFFLINE` efface ses réseaux Wi-Fi enregistrés, désactive son AP et enregistre ce choix. Elle est **optionnelle**, et ne sert pas simplement à fermer une fenêtre Web. Réponse : `OTAOFFLINE_SAVED`.

`OTARESET` redémarre la carte après ces changements. Ne pas ouvrir son port USB pendant un transfert en cours ; l'ouverture peut provoquer un reset.

Ajouter un relais

Installer le variant relais sur la carte appropriée, une seule fois :

```
export OTA_RELAY_USB='/dev/serial/by-id/PORT_USB_DU_RELAIS'
pio run -e ttgo-lora32-v21-ota-relay -t upload --upload-port "$OTA_RELAY_USB"
```

Un upload Web peut aussi convenir lorsque le bootloader et les partitions sont déjà compatibles. La nouvelle image seule ne corrige pas leur disposition.

L'API de diagnostic du relais permet de créer la route. Son accès de gestion doit être réellement joignable depuis le poste qui lance la commande :

```
export OTA_RELAY_URL='http://ADRESSE_DE_GESTION_DU_RELAIS'
export OTA_RELAY_WEB_USER='IDENTIFIANT_DE_GESTION'
python3 - <<'PY'
import os, base64
from pathlib import Path
key = Path(os.environ['OTA_ROUTE_KEY']).read_text().strip()
assert len(key) == 43 and len(base64.urlsafe_b64decode(key + '=')) == 32
path = Path(os.environ['OTA_PRIVATE']) / 'route-form.key'
path.write_text(key)
path.chmod(0o600)
PY
curl --fail --user "$OTA_RELAY_WEB_USER" \
  --data-urlencode "controller=$OTA_SOURCE" \
  --data-urlencode "target=$OTA_TARGET" \
  --data-urlencode "key@$OTA_PRIVATE/route-form.key" \
  "$OTA_RELAY_URL/lab/ota/route"
```

Curl demande le mot de passe ; celui-ci n'est pas écrit dans la commande. Les identifiants sont ceux de l'API de diagnostic : Webadmin si activé, sinon ceux de l'OTA si configurés, sinon `ota-lab` et le mot de passe de l'AP. L'enregistrement de la route **redémarre le relais**. Sans `bench_profile=1`, ses paramètres radio enregistrés sont conservés.

Sur la cible, configurer également le relais et la même clé de transport :

```
OTAROUTE F4MLV-10 <clé de transport base64url>
OTARESET
```

Réponse avant reset : `OTAROUTE_SAVED`. Une cible ainsi provisionnée exige le passage par son relais : retirer `--via` ne la transforme pas en cible directe. Inversement, une cible directe doit avoir le rôle de route désactivé. Aucune commande série `OTAROUTE=OFF` n'est implémentée.

4. Compiler l'image destinée au transfert

Cette commande compile sur le PC ; elle ne flashe aucune carte :

```
pio run -e ttgo-lora32-v21-ota-b
test -s "$OTA_IMAGE"
sha256sum "$OTA_IMAGE" | tee "$OTA_RUN/image.sha256"
```

Transmettre **firmware.bin**, pas une image fusionnée, un bootloader ou une image SPIFFS. Conserver ce fichier exact pour une éventuelle reprise. Une image dont l'en-tête ESP32 est valide mais qui ne fournit pas le service OTA/confirmation attendu ne constitue pas une mise à jour compatible.

5. Lancer un transfert complet par un relais

Fermer tout moniteur utilisant le port du pont. Le relais et la cible doivent initialement écouter le profil `normal` du JSON.

Cette commande active et confirme automatiquement l'image reçue.

```
python3 tools/lora_ota/send_integrated.py \
  --bridge "$OTA_BRIDGE" \
  --source "$OTA_SOURCE" --target "$OTA_TARGET" \
  --key-file "$OTA_KEY" --counter-file "$OTA_COUNTER" \
  --via "$OTA_VIA" --route-key-file "$OTA_ROUTE_KEY" \
  --negotiate-profile --radio-profiles "$OTA_PROFILES" \
  --windowed --block-size 128 \
  --image "$OTA_IMAGE" --log "$OTA_RUN/transfer.jsonl"
```

Ajouter `--require-offline` seulement si le Wi-Fi et l'AP de la cible ont déjà été désactivés : le client vérifie cet état avant et après la mise à jour. Il ne les désactive pas lui-même.

128 octets est la taille par défaut, déjà utilisée pour les transferts complets validés par relais. Les nouvelles tailles vont de 32 à 175 octets par relais ; les essais de 175 octets portent pour l'instant sur 8 Ko, pas sur une nouvelle image complète.

6. Variante directe

Pour une cible configurée sans relais, omettre `--via` et `--route-key-file`. Voici le mode direct historique, avec ACK par bloc :

```
python3 tools/lora_ota/send_integrated.py \
  --bridge "$OTA_BRIDGE" \
  --source "$OTA_SOURCE" --target "$OTA_TARGET" \
  --key-file "$OTA_KEY" --counter-file "$OTA_COUNTER" \
  --negotiate-profile --radio-profiles "$OTA_PROFILES" \
  --image "$OTA_IMAGE" --log "$OTA_RUN/direct.jsonl"
```

Il utilise des blocs de 204 octets. Le mode par fenêtres est aussi disponible en direct en ajoutant `--windowed --block-size 128` ; les essais matériels de l'extension directe par fenêtres ont vérifié des échantillons de 8 Ko. `--block-size` ne règle pas la taille du mode historique sans fenêtres.

7. Vérifier la fin réelle du transfert

Le client enchaîne : HELLO authentifié, négociation du profil, transfert, vérification de l'image entière, retour au profil habituel, activation, redémarrage, contrôle de la nouvelle partition et de son SHA-256, CONFIRM. La cible vérifie de nouveau son image pour la confirmation.

```
python3 - "$OTA_RUN/transfer.jsonl" <<'PY'
import json, sys
rows = [json.loads(line) for line in open(sys.argv[1])]
for row in rows:
    if row.get('event') in ('image', 'verified', 'normal_profile_confirmed',
                            'boot_pending_confirmation', 'radio_confirmed'):
        print(json.dumps(row, indent=2))
assert any(row.get('event') == 'radio_confirmed' for row in rows), 'Confirmation radio absente'
PY
```

Un simple ACK APRS, la fin d'une page, ou le compteur arrivé au nombre d'octets ne suffit pas. Le succès complet doit inclure la vérification et la confirmation du nouveau démarrage, avec une sortie du programme sans erreur. CONFIRM valide le démarrage radio et le hash, pas tous les capteurs et services fonctionnels de l'application.

`block_bench.py`, qui vérifie des préfixes de 8 ou 64 Ko sans activation, est un autre outil. Il ne remplace pas cette validation d'image complète.

8. Reprendre après interruption

Arrêter l'émetteur défaillant et laisser expirer le bail radio, jusqu'à 60 secondes après la dernière activité ; garder une petite marge avant de reprendre sur le profil habituel. Réutiliser les mêmes clés, le même compteur et **exactement le même fichier image**.

En mode `--windowed`, relancer la commande avec un **nouveau fichier de log**. WINDOW_START retrouve le point de reprise durable sur la cible : les pages complètes sont conservées, la page partielle est renvoyée. Après reset du relais, son cache perdu est rempli à nouveau par le contrôleur. Ne pas ajouter `--resume-log` : cette option est incompatible avec le mode par fenêtres et avec la négociation de profil.

Si le manifeste est différent d'un transfert conservé, la cible répond BUSY. Il faut soit reprendre l'ancienne image, soit l'abandonner explicitement avec ABORT avant de commencer la nouvelle.

9. État, abandon explicite et confirmation manuelle

Ce bloc réutilise les variables précédentes, sur le profil habituel. En direct, définir `OTA_VIA` vide ; sinon conserver le relais réellement provisionné. Aucun programme de transfert ne doit occuper le port du pont.

```
export OTA_ACTION='status'
python3 - <<'PY'
import os, sys, json, base64, hashlib
from pathlib import Path
sys.path.insert(0, 'tools/lora_ota')
from send_integrated import Client, STATUS, ABORT, CONFIRM, printable
from radio_profiles import load_profiles
from route_transport import RouteTransport
action = os.environ['OTA_ACTION']
assert action in ('status', 'abort', 'confirm')
key = base64.urlsafe_b64decode(Path(os.environ['OTA_KEY']).read_text().strip() + '=')
expected = hashlib.sha256(Path(os.environ['OTA_IMAGE']).read_bytes()).digest() if action ==
        'confirm' else None
c = Client(os.environ['OTA_BRIDGE'], key, os.environ['OTA_SOURCE'].encode(),
           os.environ['OTA_TARGET'].encode(), Path(os.environ['OTA_COUNTER']), lambda *a, **k:
        None)
try:
    if os.environ.get('OTA_VIA'):
        hop_key =
        base64.urlsafe_b64decode(Path(os.environ['OTA_ROUTE_KEY']).read_text().strip() + '=')
        c.route = RouteTransport(hop_key, c.source, c.target, os.environ['OTA_VIA'].encode())
    c.set_radio_profiles(load_profiles(Path(os.environ['OTA_PROFILES'])))
    c.enable_profiles()
    print(json.dumps(printable(c.hello(attempts=3)), indent=2))
    op = STATUS if action == 'status' else ABORT if action == 'abort' else CONFIRM
    reply = c.request(op, 0, expected or b'', attempts=3)
    print(json.dumps(printable(reply), indent=2))
finally:
    c.close()
PY
```

- `status` ouvre une session authentifiée et lit l'état ; il ne supprime pas le point de reprise. La session est ensuite laissée à son expiration.
- `abort` : remplacer `OTA_ACTION` par `abort`, puis réexécuter le bloc. Cela supprime explicitement le point de reprise et ferme la session. Cela ne réinstalle pas l'ancien firmware.
- `confirm` : remplacer `OTA_ACTION` par `confirm`, avec `OTA_IMAGE` pointant vers le fichier attendu. À utiliser si l'activation a eu lieu mais que le client s'est arrêté avant sa confirmation, tant que la cible attend cette confirmation. La fenêtre de confirmation est de 180 secondes depuis le début du setup ; en son absence le firmware demande le retour arrière. Le watchdog de démarrage peut aussi provoquer un retour avant.

10. Commandes APRS depuis Graywolf

Les envoyer comme commandes authentifiées à la bonne station avec une radio APRS opérationnelle. Le pont `bridge_fast` du banc ne fournit pas l'interface TNC2 habituelle de Graywolf.

Commande à faire signer	Effet
`OTA=?`	État OTA, armement et profil temporaire préparé
`OTA=PROFILE,434550000,250000,8,6,3`	Prépare ce profil en RAM ; ne bascule pas encore
`OTA=FAST,30`	Bascule deux secondes après la réponse, bail d'inactivité de 30 s
`OTA=NORMAL`	Répond sur le profil courant puis revient au profil habituel

Après FAST, la radio de commande doit elle aussi utiliser le profil temporaire pour joindre la station. Graywolf ne le règle pas automatiquement. Ces commandes ne transportent pas le firmware et ne coordonnent pas une bascule de plusieurs relais : le client binaire ci-dessus réalise le cycle de transfert. Ne pas précéder son lancement d'une bascule FAST manuelle.

11. Erreurs courantes et limites

Symptôme	Vérification
Aucun HELLO	Port du pont, clé, identités de huit caractères maximum, profil normal, rôle de route et présence des cartes
BUSY au début	Confirmation en attente ou manifeste d'une autre image ; ne pas effacer le point de reprise par réflexe
Refus du profil	Compatibilité de chaque radio et durée de groupe inférieure au bail ; SF5/SF6 non pris en charge par le pont SX1278
`--require-offline` échoue	La cible a encore son Wi-Fi/AP actif ; cette option vérifie mais ne configure pas
ACK d'activation absent	Vérifier le nouveau démarrage : l'ACK peut être perdu au moment du reset
Retour à l'ancien firmware	Vérifier les journaux du démarrage, le hash et la réception de CONFIRM

Ne pas utiliser pour une mise à jour normale les options `--fault-checks` et `--exercise-*` : elles injectent des pertes, resets ou retours arrière. Le client de transfert ne restaure pas les sauvegardes des cartes. Le retour au **profil radio normal** et la restauration d'un **ancien firmware** sont deux opérations différentes.

Pour les essais de tailles de blocs, garder les cartes sur le banc pendant la campagne. Une perte temporaire de l'accès de gestion ne justifie pas leur reflash. L'accès HTTP du relais sert au diagnostic/provisionnement ; il n'est pas requis par

`send_integrated.py` pour envoyer les blocs.

Références : protocole et intégration, résultats matériels, comparaison des blocs.